

Create a Blank UI for Automate from Scratch

 Information last updated 23 sept 2025	
SME	Jan Rusch
EPIC	CIC Custom User Interface and Custom Development

1	 Instructions
1.1	What You Will Build
1.2	 Custom Angular App Development Process Flow
1.3	 Prerequisites
1.4	 Why Build from Scratch Instead of Using Templates?
1.4.1	Advantages of Starting Fresh
1.4.2	When to Use Workspace Templates
1.5	 Step 1: Environment Setup
1.5.1	Install Node.js and npm
1.5.2	Install Angular CLI
1.5.3	Install Additional Tools
1.5.4	Verify Your Setup
1.6	 Step 2: Project Structure and Initialization
1.6.1	Creating the Workspace
1.6.2	Understanding the Workspace Structure
1.6.3	Creating the Main Application
1.6.4	Project Structure After Application Creation
1.7	Step 3. Configure the Bootstrap Pipeline
1.8	Step 4. Build the Shell Layout
1.9	Step 5. Establish Routing Placeholders
1.10	Step 6. Style the Blank UI
1.11	Step 7. Wire Environment-Aware Configuration
1.12	Step 8. Run and Test the Application
1.13	Step 9. Prepare for Deployment
1.14	 Video: Status tracking of eForms standard
1.15	Troubleshooting Checklist
2	 Related articles

Instructions

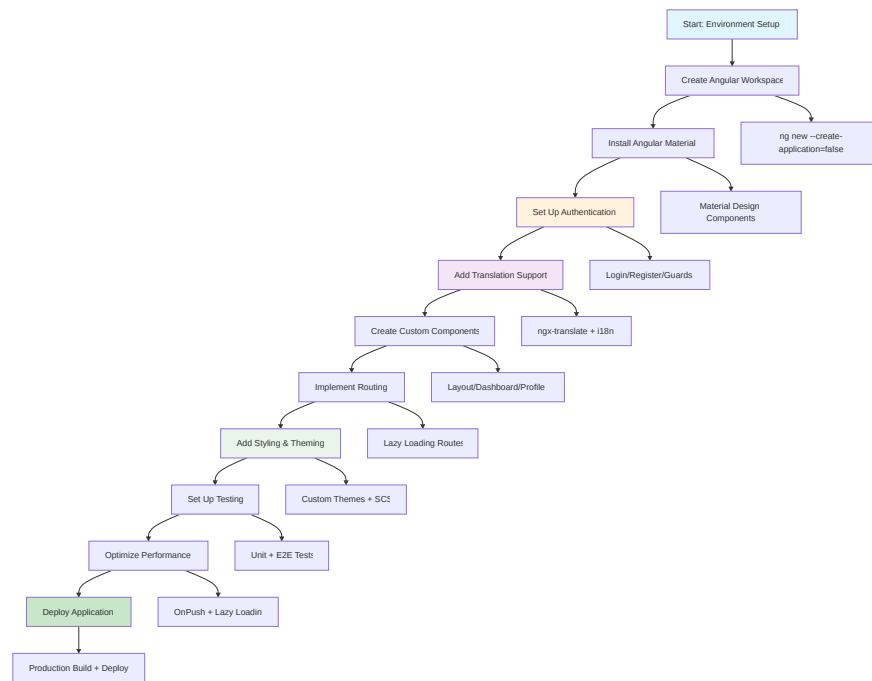
This guide shows how to assemble a focused Angular 20 workspace from the ground up. You will create just enough structure to serve a polished shell, then leave room for your product team to grow the experience deliberately.

What You Will Build

- A custom Angular workspace with a single blank application
- A routed layout anchored by a reusable shell component
- Shared styling primitives, environment-aware configuration, and typed models
- A repeatable workflow for serving locally, testing, and preparing CIC/Automate builds



Custom Angular App Development Process Flow



Prerequisites

- Node.js 18.19.0 or later (`node --version`)
- npm 9.0.0 or later (`npm --version`)
- Angular CLI 20.0.0 or later (`ng version`)
- Git for source control
- VS Code or another modern TypeScript IDE
- Working knowledge of TypeScript, HTML, SCSS, and Angular fundamentals
- Access to a CIC/Automate environment for deployment validation



Why Build from Scratch Instead of Using Templates?

Advantages of Starting Fresh

1. Complete Control and Understanding

- Every line of code is yours to understand and modify
- No hidden dependencies or legacy code to work around
- Full visibility into how each component functions
- Easier debugging and troubleshooting

2. Optimized Performance

- Only include the features you actually need
- Smaller bundle sizes and faster load times
- No unnecessary libraries consuming resources
- Better control over lazy loading and code splitting

3. Enhanced Security

- No third-party code you haven't reviewed
- Custom security implementations tailored to your needs
- Full control over authentication and authorization
- Easier to maintain security updates

4. Better Maintainability

- Clean, organized code structure
- Consistent coding patterns throughout
- Easier to onboard new developers
- Simplified testing and debugging

5. Custom Architecture

- Design patterns that fit your specific use case
- Scalable folder structure
- Custom services and utilities
- Tailored state management approach

When to Use Workspace Templates

Workspace templates are still valuable when:

- Standard features meet your requirements
-

Step 1: Environment Setup

Install Node.js and npm

1. **Download Node.js** from the official website
2. **Verify Installation:**

```
1 node --version
```

```
2 npm --version
```

```
PS C:\Users\jrusch\Documents\CICI DOCUMENTATION UI UPDATER>
PS C:\Users\jrusch\Documents\CICI DOCUMENTATION UI UPDATER> node --version
v20.19.3
PS C:\Users\jrusch\Documents\CICI DOCUMENTATION UI UPDATER>
PS C:\Users\jrusch\Documents\CICI DOCUMENTATION UI UPDATER>
PS C:\Users\jrusch\Documents\CICI DOCUMENTATION UI UPDATER>
PS C:\Users\jrusch\Documents\CICI DOCUMENTATION UI UPDATER> npm --version
11.4.2
PS C:\Users\jrusch\Documents\CICI DOCUMENTATION UI UPDATER>
PS C:\Users\jrusch\Documents\CICI DOCUMENTATION UI UPDATER>
```

Install Angular CLI

```
1 npm install -g @angular/cli@20
```

Verify Installation:

```
1 ng version
```

```
PS C:\Users\jrusch\Documents\CICI DOCUMENTATION UI UPDATER> ng version

Angular CLI
o
Angular CLI: 20.1.0
Node: 20.19.3
Package Manager: npm 11.4.2
OS: win32 x64

Angular:
...

Package      Version
-----
@angular-devkit/architect    0.2001.0 (cli-only)
@angular-devkit/core         20.1.0 (cli-only)
@angular-devkit/schematics   20.1.0 (cli-only)
@schematics/angular          20.1.0 (cli-only)
```

Install Additional Tools

```
1 # Install TypeScript globally (optional but recommended)
2 npm install -g typescript
3
4 # Install Angular DevKit (for advanced configurations)
5 npm install -g @angular-devkit/schematics-cli
6
7 # Install Angular Material CLI (for component generation)
8 npm install -g @angular/material
```

Verify Your Setup

Create a test workspace to ensure everything works:

```
1 ng new test-app --create-application=false
```

```

PS C:\Users\jrusch\Documents\TemplateCICDocumentation> ng new test-app --create-application=false
✓ Do you want to create a 'zoneless' application without zone.js (Developer Preview)? No
CREATE test-app/angular.json (146 bytes)
CREATE test-app/package.json (1069 bytes)
CREATE test-app/README.md (1529 bytes)
CREATE test-app/tsconfig.json (897 bytes)
CREATE test-app/.editorconfig (331 bytes)
CREATE test-app/.gitignore (629 bytes)
CREATE test-app/.vscode/extensions.json (134 bytes)
CREATE test-app/.vscode/launch.json (490 bytes)
CREATE test-app/.vscode/tasks.json (980 bytes)
✓ Packages installed successfully.
  Successfully initialized git.
PS C:\Users\jrusch\Documents\TemplateCICDocumentation>

```

```

1 cd test-app
2 ng generate application test-app

```

```

PS C:\Users\jrusch\Documents\TemplateCICDocumentation\test-app> ng generate application test-app
✓ Which stylesheet format would you like to use? CSS [ https://developer.mozilla.org/docs/Web/CSS ]
✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No
✓ Do you want to create a 'zoneless' application without zone.js? No
CREATE projects/test-app/src/app/app.spec.ts (689 bytes)
CREATE projects/test-app/src/app/app.ts (302 bytes)
CREATE projects/test-app/src/app/app.css (0 bytes)
CREATE projects/test-app/src/app/app.html (20464 bytes)
CREATE projects/test-app/src/main.ts (228 bytes)
CREATE projects/test-app/src/app/app.config.ts (412 bytes)
CREATE projects/test-app/src/app/app.routes.ts (80 bytes)
CREATE projects/test-app/tsconfig.app.json (452 bytes)
CREATE projects/test-app/tsconfig.spec.json (430 bytes)
CREATE projects/test-app/public/favicon.ico (15086 bytes)
CREATE projects/test-app/src/index.html (386 bytes)
CREATE projects/test-app/src/styles.css (81 bytes)
UPDATE angular.json (2655 bytes)
UPDATE tsconfig.json (1062 bytes)
UPDATE package.json (1083 bytes)
✓ Packages installed successfully.

```

```

1 ng serve

```

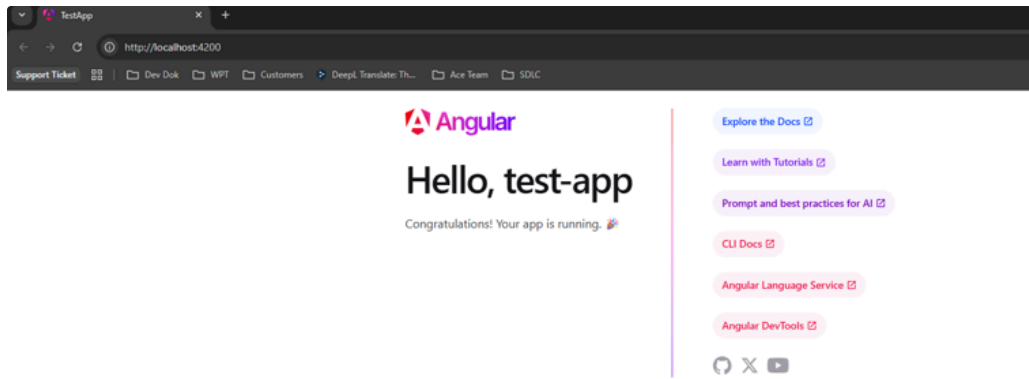
```

PS C:\Users\jrusch\Documents\TemplateCICDocumentation\test-app>
PS C:\Users\jrusch\Documents\TemplateCICDocumentation\test-app>
PS C:\Users\jrusch\Documents\TemplateCICDocumentation\test-app> ng serve
Initial chunk files | Names | Raw size |
main.js | main | 48.63 kB |
styles.css | styles | 113 bytes |
polyfills.js | polyfills | 95 bytes |
| Initial total | 48.84 kB |

Application bundle generation complete. [1.866 seconds] - 2025-09-23T10:58:01.465Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
→ Local: http://localhost:4200/
→ press h + enter to show help

```



🔧 Step 2: Project Structure and Initialization

Creating the Workspace

Unlike the standard `ng new` command, we'll create a workspace without the default application:

```
1 | ng new my-custom-app --create-application=false --routing=true --style=scss --package-manager=r
```

```
PS C:\Users\jrusch\Documents\TemplateCICDocumentation>
PS C:\Users\jrusch\Documents\TemplateCICDocumentation>
PS C:\Users\jrusch\Documents\TemplateCICDocumentation>
PS C:\Users\jrusch\Documents\TemplateCICDocumentation> ng new my-custom-app --create-application=false --routing=true --style=scss --package-manager=rm
✓ Do you want to create a 'zoneless' application without zone.js (Developer Preview)? No
CREATE my-custom-app/angular.json (193 bytes)
CREATE my-custom-app/package.json (1874 bytes)
CREATE my-custom-app/README.md (1533 bytes)
CREATE my-custom-app/tsconfig.json (897 bytes)
CREATE my-custom-app/.editorconfig (331 bytes)
CREATE my-custom-app/.gitignore (629 bytes)
CREATE my-custom-app/.vscode/extensions.json (134 bytes)
CREATE my-custom-app/.vscode/launch.json (498 bytes)
CREATE my-custom-app/.vscode/tasks.json (980 bytes)
✓ Packages installed successfully.
Successfully initialized git.
PS C:\Users\jrusch\Documents\TemplateCICDocumentation>
```

This command creates:

- A workspace configuration file (`angular.json`)
- Package management files (`package.json` , `package-lock.json`)
- TypeScript configuration (`tsconfig.json`)
- Git repository initialization
- SCSS support for styling

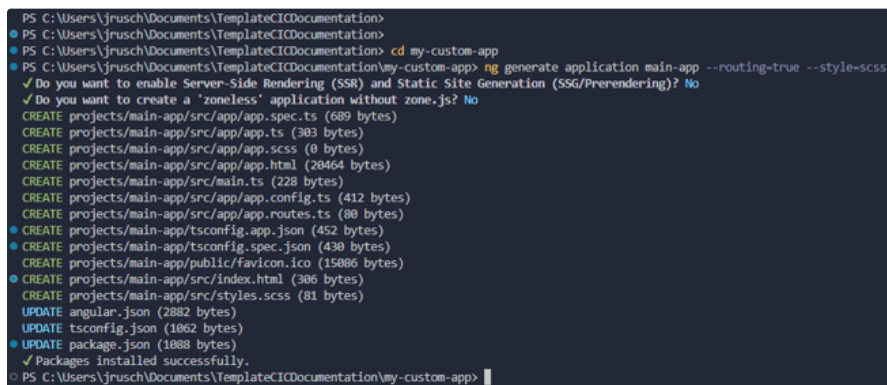
Understanding the Workspace Structure

```
1 my-custom-app/  
2 |— angular.json           # Workspace configuration  
3 |— package.json           # Dependencies and scripts  
4 |— package-lock.json      # Dependency lock file  
5 |— tsconfig.json          # TypeScript configuration  
6 |— .gitignore             # Git ignore rules  
7 |— .editorconfig          # Editor configuration  
8 |— README.md              # Project documentation
```

Creating the Main Application

Navigate to your workspace and create the main application:

```
1 cd my-custom-app  
2 ng generate application main-app --routing=true --style=scss
```



```
PS C:\Users\jrusch\Documents\TemplateCICDocumentation>  
PS C:\Users\jrusch\Documents\TemplateCICDocumentation>  
PS C:\Users\jrusch\Documents\TemplateCICDocumentation> cd my-custom-app  
PS C:\Users\jrusch\Documents\TemplateCICDocumentation\my-custom-app> ng generate application main-app --routing=true --style=scss  
✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No  
✓ Do you want to create a 'zoneless' application without zone.js? No  
CREATE projects/main-app/src/app/app.spec.ts (689 bytes)  
CREATE projects/main-app/src/app/app.ts (303 bytes)  
CREATE projects/main-app/src/app/app.scss (0 bytes)  
CREATE projects/main-app/src/app/app.html (20464 bytes)  
CREATE projects/main-app/src/main.ts (228 bytes)  
CREATE projects/main-app/src/app/app.config.ts (412 bytes)  
CREATE projects/main-app/src/app/app.routes.ts (80 bytes)  
CREATE projects/main-app/tsconfig.app.json (452 bytes)  
CREATE projects/main-app/tsconfig.spec.json (430 bytes)  
CREATE projects/main-app/public/favicon.ico (15086 bytes)  
CREATE projects/main-app/src/index.html (306 bytes)  
CREATE projects/main-app/src/styles.scss (81 bytes)  
UPDATE angular.json (2882 bytes)  
UPDATE tsconfig.json (1062 bytes)  
UPDATE package.json (1088 bytes)  
✓ Packages installed successfully.  
PS C:\Users\jrusch\Documents\TemplateCICDocumentation\my-custom-app>
```

This creates a complete Angular application with:

- Standalone component structure
- Modern routing configuration
- SCSS styling support
- Testing setup
- Application configuration (app.config.ts)

Project Structure After Application Creation

```
1 my-custom-app/  
2 |— angular.json  
3 |— package.json  
4 |— projects/  
5 |   |— main-app/  
6 |       |— public/  
7 |           |— favicon.ico  
8 |       |— src/  
9 |           |— app/  
10 |               |— app.html  
11 |               |— app.scss  
12 |               |— app.spec.ts  
13 |               |— app.ts  
14 |               |— app.config.ts  
15 |               |— app.routes.ts  
16 |               |— index.html
```

```

17 |     |   | main.ts
18 |     |   | styles.scss
19 |     |   | tsconfig.app.json
20 |     |   | tsconfig.spec.json
21 |     |   | ...

```

Step 3. Configure the Bootstrap Pipeline

1. Open `projects/main-app/src/main.ts` and keep the bootstrap minimal:

```

1 import { bootstrapApplication } from '@angular/platform-browser';
2 import { provideRouter } from '@angular/router';
3 import { AppComponent } from './app/app';
4 import { routes } from './app/app.routes';
5
6 bootstrapApplication(AppComponent, {
7   providers: [provideRouter(routes)],
8 }).catch(err => console.error(err));

```

1. Add data providers, telemetry, or state tools only after the UI shell stabilizes.

Step 4. Build the Shell Layout

1. Replace `projects/main-app/src/app/app.html` with a compact layout:

```

1 <header class="app-header">
2   <h1>My Custom Application</h1>
3   <nav>
4     <a routerLink="/" routerLinkActive="active" ariaCurrentWhenActive="page">Home</a>
5     <a routerLink="/about" routerLinkActive="active" ariaCurrentWhenActive="page">About</a>
6   </nav>
7 </header>
8
9 <main class="app-content">
10   <router-outlet></router-outlet>
11 </main>
12
13 <footer class="app-footer">@ {{ currentYear }} My Company</footer>

```

1. Surface template data from `projects/main-app/src/app/app.ts`:

```

1 import { Component, computed, signal } from '@angular/core';
2
3 @Component({
4   selector: 'app-root',
5   standalone: true,
6   templateUrl: './app.html',
7   styleUrls: ['./app.scss'],
8 })
9 export class AppComponent {
10   readonly currentYear = new Date().getFullYear();
11   readonly title = signal('My Custom Application');
12   readonly banner = computed(() => `${this.title()} Starter`);
13 }

```

Step 5. Establish Routing Placeholders

1. Author basic routes in `projects/main-app/src/app/app.routes.ts`:


```

1 import { Routes } from '@angular/router';
2
3 export const routes: Routes = [
4   {
5     path: '',
6     loadComponent: () => import('./features/home/home.component').then(m => m.HomeComponent),
7   },
8   {
9     path: 'about',
10    loadComponent: () => import('./features/about/about.component').then(m => m.AboutComponent),
11  },
12  { path: '**', redirectTo: '', pathMatch: 'full' },
13 ];

```

1. Generate feature placeholders:

```

1 ng g component projects/main-app/src/app/features/home --standalone --skip-tests=false
2 ng g component projects/main-app/src/app/features/about --standalone --skip-tests=false

```

1. Keep components standalone to simplify lazy loading and cut module overhead.

Step 6. Style the Blank UI

1. Define global tokens in `projects/main-app/src/styles.scss`:

```

1 :root {
2   --app-max-width: 1080px;
3   --app-spacing: 1.5rem;
4   font-family: 'Inter', system-ui, sans-serif;
5 }
6
7 *, *::before, *::after {
8   box-sizing: border-box;
9 }
10
11 body {
12   margin: 0;
13   background-color: #f6f8fb;
14   color: #1a1a1a;
15 }

```

1. Scope shell-specific styling inside `projects/main-app/src/app/app.scss`:

```

1 .app-header,
2 .app-footer {
3   padding: var(--app-spacing);
4   background-color: #fff;
5   border-bottom: 1px solid #e0e4eb;
6 }
7
8 .app-content {
9   margin: 0 auto;
10  max-width: var(--app-max-width);
11  padding: var(--app-spacing);
12 }
13
14 nav a {
15   margin-right: 1rem;
16   text-decoration: none;
17   color: #0050d4;
18 }
19
20 nav a.active {

```

```
21 font-weight: 600;
22 border-bottom: 2px solid #0050d4;
23 }
```

1. Store feature-level styles next to each component to keep concerns isolated.

Step 7. Wire Environment-Aware Configuration

1. Model application settings in `projects/main-app/src/app/models/app-config.model.ts`:

```
1 export interface AppConfig {
2   environment: 'local' | 'test' | 'prod';
3   apiUrl: string;
4 }
```

1. Provide runtime defaults in `projects/main-app/src/public/app.config.json`:

```
1 {
2   "environment": "local",
3   "apiUrl": "http://localhost:3000"
4 }
```

1. Use the initializer in `app.config.ts` to load this JSON before Angular renders, enabling configuration changes without a rebuild.

Step 8. Run and Test the Application

1. Install dependencies and run the dev server:

```
1 npm install
2 ng serve main-app --open
```

1. Execute the default unit test suite:

```
1 ng test main-app
```

1. Add component tests around the shell and feature placeholders to guard future refactors.

Step 9. Prepare for Deployment

1. Duplicate the production configuration in `angular.json` for each CIC stage and adjust replacements or budgets as required.
2. Produce an optimized bundle:

```
1 ng build main-app --configuration=production
```

1. Publish `dist/main-app` to your CIC/Automate platform and include the runtime `app.config.json` expected by your environment (With Placeholder which get replaced by the Build Process from the CIC Platform).

📺 Video: Status tracking of eForms standard

Click to sign in and play video



process-statu... ain.zip
01 abr 2026, 07:15 am

Troubleshooting Checklist

- `npm start` or `ng serve` fails immediately: remove `node_modules`, run `npm install`, and retry.
- Routes return blank screens: confirm `provideRouter(routes)` is registered and the route paths match your links.
- Styling disappears in production: ensure component styles are referenced and global styles are listed in `angular.json` assets.
- Configuration does not load: verify `app.config.json` is part of the `assets` array and the initializer awaits its fetch.

📋 Related articles